
Secure Packages with CodeArtifact

Danford Amos

	Package name	Namespace	Format	Latest version	Latest publish date	Publish	Upstream
☐	backport-util-concurrent	backport-util-concurrent	maven	3.1	18 minutes ago	Block	Allow
☐	classworlds	classworlds	maven	1.1	19 minutes ago	Block	Allow
☐	google	com.google	maven	1	18 minutes ago	Block	Allow
☐	jsr305	com.google.code.findbugs	maven	2.0.1	18 minutes ago	Block	Allow
☐	google-collections	com.google.collections	maven	1.0	18 minutes ago	Block	Allow
☐	commons-cli	commons-cli	maven	1.0	19 minutes ago	Block	Allow
☐	commons-	commons-	maven	1.1	18 minutes	Block	Allow

Introducing Today's Project!

Key tools and concepts

Through this project, I strengthened my skills in EC2 server administration by installing software, managing packages, navigating Linux, and configuring the environment for Java and Maven builds directly on a cloud instance. I also gained hands-on experience with IAM authentication, using access keys, AWS CLI configuration, and permissions to securely authorize both the EC2 instance and Maven to interact with AWS services. In addition, I deepened my understanding of version control with Git by committing, pushing, pulling, and managing my codebase through GitHub as the remote repository. This included practical repository integration work such as connecting EC2 to GitHub, cloning the project, and keeping the environment synchronized. Altogether, these experiences strengthened my grasp of cloud infrastructure fundamentals—specifically how compute, identity, and source control work together to support a cloud-based development workflow.

Project reflection

It took you a few focused hours to complete this project from start to finish. That time included launching and configuring your EC2 instance, setting up IAM authentication, cloning your project from GitHub, integrating with CodeArtifact, compiling your web app, publishing your own custom package, and finally validating that package by downloading and inspecting it in CloudShell. Across all those steps, the work was concentrated, hands-on, and completed within the same session—showing that you now have a solid end-to-end understanding of how compute, identity, source control, and artifact management fit together in AWS.

This project is part three of a series of DevOps projects where I'm building a CI/CD pipeline! I'll be working on the next project.

CodeArtifact Repository

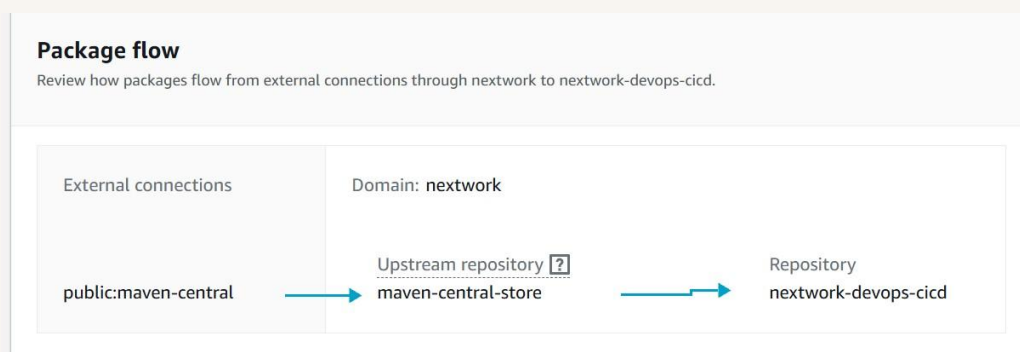
AWS CodeArtifact is a managed artifact repository service that integrates directly with CI/CD pipelines to provide secure, consistent, and centralized package management. Pipelines use CodeArtifact to pull approved dependencies, cache external packages, and publish versioned build artifacts, ensuring reliable builds, strong supply-chain security, and a controlled flow of software components across development, testing, and deployment stages.

Danford Amos

Engineering teams use CodeArtifact for security, control and reliability.

A CodeArtifact domain is a top-level grouping that organizes multiple repositories under a single security, governance, and storage boundary. It allows us to manage permissions, share packages, and maintain consistent CI/CD workflows across all repositories. Our domain is named `nextwork`.

A CodeArtifact repository can be connected to an upstream repository, which is a public package source that Maven checks when a dependency isn't found locally. Our repository's upstream is Maven Central, the largest Java package repository. By linking CodeArtifact to Maven Central, we get full access to public Java libraries while gaining the benefits of caching, security, and consistent builds inside our private CodeArtifact environment.



CodeArtifact Security

Issue

In this step, we connect Maven to our CodeArtifact repository. Maven is our web app's package manager, and CodeArtifact is where those packages are stored. We need an authorization token to access CodeArtifact because it is a private, IAM-protected repository. The token gives our EC2 instance temporary (12-hour) permission to download packages. Initially, we ran into errors because the EC2 instance had no IAM role attached, meaning it had no permissions to request a token. This follows AWS's principle of least privilege, where resources start with zero access until explicitly granted.

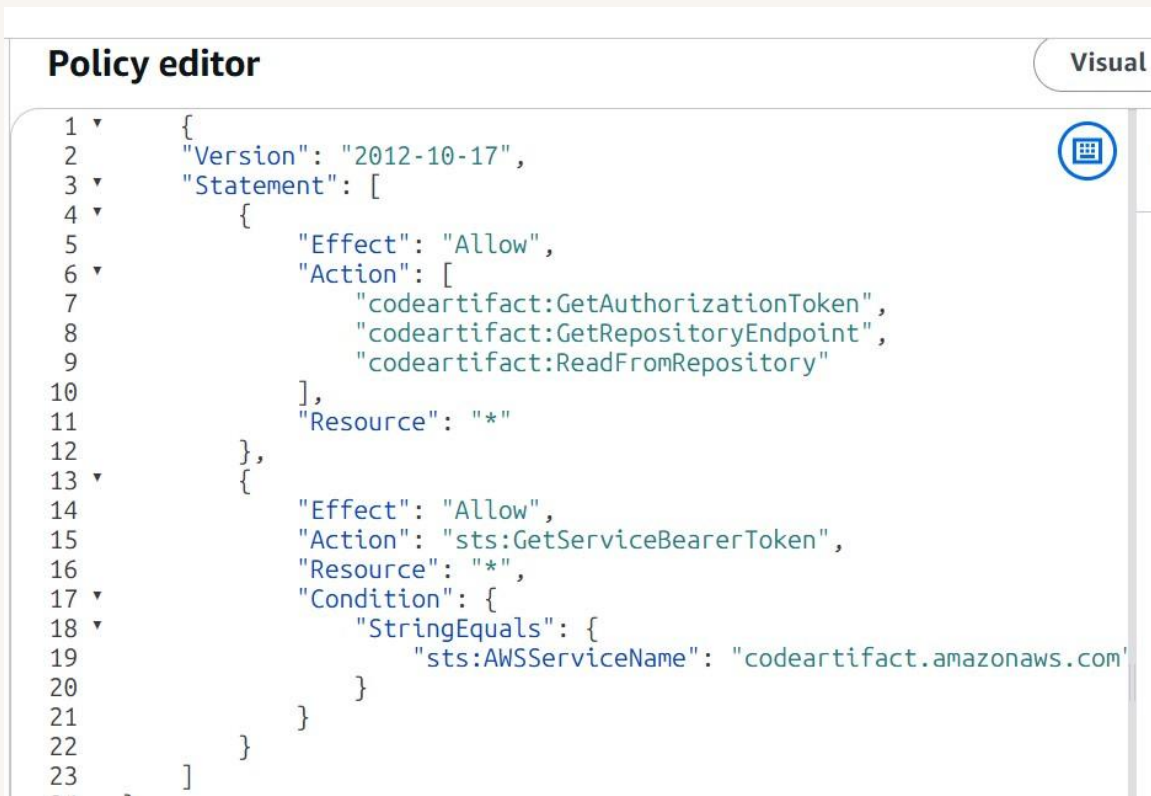
Resolution

We resolved the permissions error by creating an IAM policy that grants access to CodeArtifact and attaching it to an IAM role assigned to our EC2 instance. Once the role was applied, the instance gained the permissions needed to request an authorization token and connect to our repository, which fixed the authentication error caused by the EC2 instance having no default access.

Using IAM roles is a security best practice because roles provide temporary, automatically rotated credentials without requiring any hardcoded access keys. Hardcoded credentials are vulnerable to leaks and misuse, while IAM roles follow the principle of least privilege and allow us to securely grant the same permissions to multiple EC2 instances in a scalable, controlled way.

The JSON policy attached to my role

This IAM policy grants the minimum permissions required for our EC2 instance to authenticate to CodeArtifact and download packages. It allows the instance to retrieve an authorization token, discover the repository endpoint, and read packages from the repository. These permissions are necessary because CodeArtifact is private, and our EC2 instance starts with no access by default, following the principle of least privilege.



Maven and CodeArtifact

To test the connection between Maven and CodeArtifact, I compiled my web app using settings.xml

The settings.xml file configures Maven to use our CodeArtifact repository. It provides Maven with the repository name, the authentication token needed to access CodeArtifact, and a profile that tells Maven when to use this repository— especially useful when multiple repositories exist. This setup ensures Maven can securely retrieve all required packages from our private CodeArtifact repository.

Compiling is the process of translating our webapp's source code into machine-readable bytecode. Maven performs this compilation, and during the process it must gather all required packages. Maven first checks our CodeArtifact repository for these dependencies. If CodeArtifact doesn't have them, it retrieves them from its upstream repository (Maven Central) and stores a local copy. This ensures our builds are consistent, secure, and repeatable.

```
settings.xml
1  <settings>
2  <servers>
3    <server>
4      <id>nextwork-nextwork-devops-cicd</id>
5      <username>aws</username>
6      <password>${env.CODEARTIFACT_AUTH_TOKEN}</password>
7    </server>
8  </servers>
9
10 <profiles>
11   <profile>
12     <id>nextwork-nextwork-devops-cicd</id>
13     <activation>
14       <activeByDefault>true</activeByDefault>
15     </activation>
16     <repositories>
17       <repository>
18         <id>nextwork-nextwork-devops-cicd</id>
19         <url>https://nextwork-171414227038.d.codeartifact.us-east-1.amazonaws.com/maven/nex
20       </repository>
21     </repositories>
22   </profile>
23 </profiles>
24
25 <mirrors>
```

Verify Connection

After compiling your web app, you opened your CodeArtifact repository and saw four full pages of packages. That means Maven successfully downloaded your dependencies through CodeArtifact and CodeArtifact cached/stored them inside your private repository.

	Package name	Namespace	Format	Latest version	Latest publish date	Publish	Upstream
☐	backport-util-concurrent	backport-util-concurrent	maven	3.1	18 minutes ago	Block	Allow
☐	classworlds	classworlds	maven	1.1	19 minutes ago	Block	Allow
☐	google	com.google	maven	1	18 minutes ago	Block	Allow
☐	jsr305	com.google.code.findbugs	maven	2.0.1	18 minutes ago	Block	Allow
☐	google-collections	com.google.collections	maven	1.0	18 minutes ago	Block	Allow
☐	commons-cli	commons-cli	maven	1.0	19 minutes ago	Block	Allow
☐	commons-	commons-	maven	1.1	18 minutes	Block	Allow

Uploading My Own Packages

As an extension of the project, I also took on the role of a package publisher by learning how to publish custom packages into our CodeArtifact repository. This capability is valuable in real development environments where internal teams create shared libraries, utilities, or components that need to be distributed privately. By publishing our own packages, we enable internal collaboration— team members can access and consume shared code without exposing it publicly or granting unnecessary permissions to external users. This approach strengthens security, improves dependency management, and ensures that only authorized developers can access internal packages while still maintaining a smooth and centralized workflow for sharing team-built software.

After publishing my package using the CLI upload command, CodeArtifact displays a detailed view of the artifact. In the package details page, I can see the version number, which confirms the specific release I uploaded; the published date, showing exactly when the package was pushed to the repository; and the origin information, which in this case identifies the package

Danford Amos

as one that I created and uploaded myself. These details verify that the package was successfully published, recognized as an internally produced artifact, and is now available for other authorized team members to consume within our private CodeArtifact repository.

To validate my package, I downloaded it back into CloudShell directly from CodeArtifact to confirm that it could be retrieved and installed successfully. The download completed without errors, showing that CodeArtifact recognized and stored the package correctly. After that, I unzipped the archive and inspected the contents. Inside, I could clearly see the packaged files—including the original text file I created—exactly as they were before publishing. This end-to-end check confirmed that CodeArtifact securely stored my custom package, preserved its integrity, and allowed it to be retrieved later just like any other internal dependency.

```
us-east-1 +
STATUS: PUBLISHED,
--asset-sha256 $ASSET_SHA256
{
  "format": "generic",
  "namespace": "secret-mission",
  "package": "secret-mission",
  "version": "1.0.0",
  "versionRevision": "Qd+H9Be330nnvHnbHnj976kMtP8y69CA5yOMWlkW0dk=",
  "status": "Published",
  "asset": {
    "name": "secret-mission.tar.gz",
    "size": 168,
    "hashes": {
      "MD5": "a5ac1fd17928b93f7288307c2d4a34af",
      "SHA-1": "2ff4efc85663d3a37b4fb7e4e10de2e5a998d7b0",
      "SHA-256": "09a828655c5af7deeffb95b8574c94b530a0eef286c973a1312ad8d832f36f28",
      "SHA-512": "73c5964715f04649bf8bf3328fb53be530f0ba00fcd42d3662a3885facc6d91801e11f7f2e25ca0175801841628c54237a04469ed718e952db2b949856a2260e"
    }
  }
}
```
